DFSha — Sistema de Archivos Distribuido con Alta Disponibilidad, Rendimiento y Seguridad

Informe técnico completo · Proyecto 1 · Sistemas Distribuidos 2026-2

Juan Sebastián Jaramillo García **Gloria Yuliana Peña Ibargüen** **Juan José Álvarez Restrepo**

Universidad Pontificia Bolivariana — Facultad de Ingeniería, Arquitectura de Nube y Sistemas Distribuidos · Profesor: Álvaro Enrique Ospina Sanjuan · 8 de noviembre de 2026

Índice

1. Matriz de riesgo 2

2. Resumen ejecutivo 3
3. Introducción 3
4. Alcance del proyecto 4
5. Diseño original (Fase I individual — Gloria Yuliana Peña Ibargüen) 4
6. Arquitectura consensuada final 5
7. Evolución del diseño 7
8. Diseño e implementación 9
9. APIs y comunicación 10
10. Algoritmos y lógica distribuida 12
11. Despliegue 13
12. Estrategia de pruebas 14
13. Resultados experimentales 15
14. Decisiones de ingeniería 17
15. Limitaciones 17
16. Trabajo futuro 18
17. Conclusiones 18

Opción arquitectónica: Opción 1 — Cliente/servidor con servicio distribuido entre servidores (S2S), en red privada de AWS Academy

1. Matriz de Riesgo

Criterio	Pregunta que responde	Peso propuesto
Buen diseño	¿Cumple RF1–RF3 y RNF1–RNF7, con garantías claras y sin fallas conceptuales?	30 %
Alcanzable	¿Tres personas pueden implementarlo y probarlo en la Fase II? (número de piezas, piezas desconocidas, esfuerzo)	40 %
Viable	¿Corre en AWS Academy (VM, red, límites del laboratorio) y se opera sin servicios pesados?	30 %

Dimensión	Gloria	J Sebastián	Juan José	ADR
Topología	3 MDS, ≥ 5 DN, NGINX y DNS, 3 AZ	3 MDS con Consul, ≥ 4 DN, Keycloak, Vault, Prometheus, Grafana y Loki; 9 VM en 2 AZ	3 controladores, ≥ 5 DN, 2 HAProxy y observabilidad; 12 VM en 2 AZ (controladores 2 + 1)	01
Consenso de metadatos	Raft; ubicaciones de réplicas dentro del estado Raft	Raft; mapa bloque a DN en memoria, reconstruido con reportes	Igual que J Sebastián	02, 03
Tamaño de bloque	64 MiB (4 MiB en pruebas)	64 MiB (8 a 128)	16 MiB (4 a 64)	05
Replicación y colocación	R = 3, W = 2; *power of two choices*, ≥ 2 zonas	R = 3, W = 2 en cadena; 1.ª réplica en zona A y 2.ª y 3.ª en zona B	R = 3, W = 2 en pipeline; *power of two choices*, ≥ 2 AZ	06, 07
Protocolos	REST (control) y gRPC (todo el plano de datos)	gRPC en todo, con pasarela REST para pruebas	REST (control), HTTPS cliente-DN, gRPC entre servidores	08
Semántica de write()	Modos x, w, a, r+; copy-on-write por bloque	w, a, rw y truncate; copy-on-write	R, W_FILE, W_RANGE y APPEND; copy-on-write	10
Concurrencia y locks	CAS por versión de archivo con *rebase* si los bloques son disjuntos; lock opcional (lease 30 s, época)	CAS por bloque; lock asesor por rango (lease 30 s, fencing)	CAS por bloque; lock obligatorio con cola FIFO, multirango atómico y fencing	11, 12
Escritura interrumpida	Transacción OPEN, PREPARED, COMMITTED o ABORTED; TTL 60 s; .part a .blk	Sesión de 60 s; tmp/ a finalized/; UpdatePipeline retoma desde el último paquete	.tmp con rename; recibos firmados; reasigna con block_id nuevo	09, 14
Detección de fallos	Heartbeat 3 s; SUSPECT 10 s; DEAD 30 s	Heartbeat 3 s; SUSPECT 10 s; DEAD 60 s	Heartbeat 3 s; SUSPECT 9 s; DEAD 30 s	15

Descubrimiento (RNF7)	DNS y NGINX; 307 al líder; manifiesto con réplicas	DNS con Consul; leaderHint	DNS y HAProxy; NOT_LEADER; caché de 60 s	16
Autenticación y autorización	Argon2id y JWT EdDSA en el MDS; token de bloque Ed25519 de 60 s	Keycloak (OIDC) y JWT RS256; token de bloque HMAC de 10 min	Argon2id y JWT Ed25519; capacidad Ed25519 de hasta 5 min	17
Cifrado en reposo	Cliente; DEK por archivo envuelta con KEK (archivo 0400 o KMS); nonce aleatorio y AAD	AES-GCM por bloque; DEK envuelta con clave por usuario en Vault; LUKS en MDS	Cliente; DEK por archivo envuelta con KEK; nonce determinista y AAD	18
Comunicación interna	mTLS con CA privada; grupos de seguridad por puerto	mTLS con PKI de Vault (24 h)	mTLS con CA interna (24 h); grupos de seguridad	18
Experimentos y documentación	E1 a E10 con CSV; anexo de APIs y errores	E1 a E7 con dfs-bench	E1 a E8; diagramas por escenario; garantías G1 a G6	19

Evaluación por criterio

Propuesta	Buen diseño	Alcanzable	Viable	Total ponderado (30/40/30)	Total con pesos iguales
Gloria	4	4	4	**4,0**	4,0
J Sebastián	3	2	2	**2,3**	2,3
Juan José	5	3	3	**3,6**	3,7

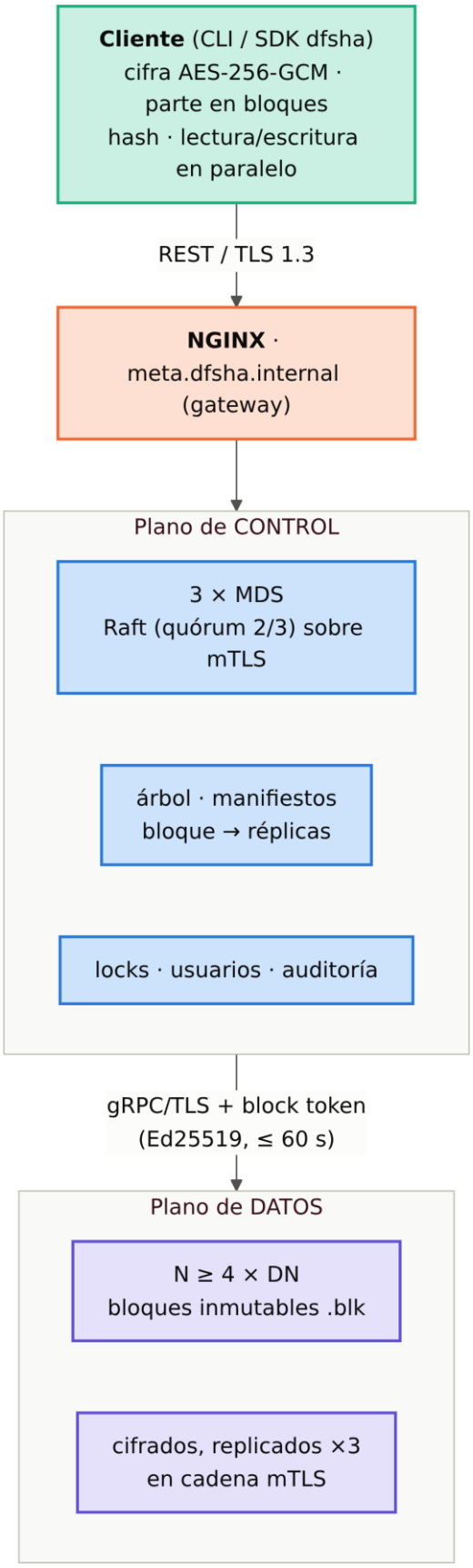


Figura 1: Arquitectura general de DFSha: cliente, gateway, plano de control (MDS/Raft) y plano de datos (DN)

2. Resumen ejecutivo

Problema. Un sistema de archivos tradicional, de un solo servidor, no escala en capacidad ni en disponibilidad: si el disco se llena o el servidor cae, el servicio se detiene por completo. DFSha resuelve esto particionando cada archivo en bloques que se reparten entre varios nodos, replicando cada bloque tres veces en zonas de disponibilidad distintas, y separando la decisión de "dónde está cada cosa" (plano de control, replicado con consenso Raft) de "quién mueve los bytes" (plano de datos, que habla directo con el cliente).

Objetivo del sistema. Implementar y verificar, con procesos reales comunicándose por red (no una simulación en memoria), un sistema de archivos distribuido que cumpla los siete requisitos no funcionales del enunciado — escalabilidad, alta disponibilidad, consistencia, particionamiento, rendimiento bajo concurrencia, seguridad y transparencia de localización
— y los tres requisitos funcionales (gestión jerárquica, transferencia de archivos, acceso por primitivas open/read/write/lock).

Principales resultados. El prototipo corre en 8 máquinas virtuales reales de AWS Academy (3 nodos de metadatos, 4 nodos de datos, 1 *gateway*), en 3 zonas de disponibilidad distintas, con TLS 1.3 y mTLS en todas las conexiones. Las 7 pruebas mínimas exigidas por el enunciado (§5.2) pasan contra ese clúster real, incluyendo matanza real de procesos (kill -9), partición de red real (iptables) y corrupción real de un bloque en disco — no simulaciones. El sistema sostiene 71–110 MB/s de transferencia por cliente y recupera la replicación completa de un nodo caído en minutos, sin perder ni un byte y sin publicar nunca un archivo a medias.

Tecnologías. Python 3.13 · FastAPI (REST de control) · gRPC/HTTP2 (plano de datos) · PySyncObj (Raft) · cryptography (AES-256-GCM, Ed25519, EdDSA) · Argon2id · Docker · NGINX · AWS Academy (EC2, EBS cifrado, VPC).

---

3. Introducción

Contexto. El curso pide diseñar e implementar, en equipo, un sistema de archivos distribuido real: no un prototipo que simule varios nodos en el mismo proceso, sino máquinas separadas que se hablan por la red, con las garantías de un sistema distribuido genuino (tolerancia a fallos, consistencia definida, seguridad de extremo a extremo).

Motivación. Casi cualquier aplicación moderna de escala necesita almacenar archivos que un solo disco, o incluso un solo servidor, no puede manejar con la disponibilidad y el rendimiento requeridos. Sistemas como GFS, HDFS o AFS resolvieron este problema hace dos décadas separando metadatos de datos y aceptando compromisos explícitos entre consistencia, disponibilidad y rendimiento (el triángulo que organiza buena parte de este informe). DFSha es una versión reducida y didáctica de esa misma idea, construida desde cero por el equipo.

Problema que se busca resolver. Concretamente: ¿cómo se reparte un archivo grande entre varios nodos sin perder la capacidad de recuperarlo exactamente igual; cómo se detecta y se recupera un nodo caído sin que el usuario note un archivo corrupto o a medias; y cómo se garantiza que dos escritores concurrentes sobre el mismo archivo no se pisen en silencio?

Importancia del enfoque distribuido. Un enfoque centralizado resolvería estas preguntas trivialmente (un único punto de verdad), pero al costo de un único punto de falla y un techo de capacidad fijo. El reto real — y el que este proyecto exige demostrar con código funcionando en red — es sostener esas mismas garantías cuando la verdad vive repartida en

varios procesos que pueden fallar de forma independiente.

---

4. Alcance del proyecto

Qué resuelve el sistema.

- Almacenamiento jerárquico privado por usuario (/users/<uid>/...), con autenticación y autorización.
- Carga y descarga de archivos de cualquier tamaño, particionados automáticamente en bloques de 64 MiB.
- Acceso de grano fino: abrir, leer un rango de bytes, escribir por rangos o anexar, sin descargar el archivo completo.
- Tolerancia real a la caída de un nodo de datos o del líder de metadatos, sin pérdida de datos confirmados.
- Cifrado de extremo a extremo: el contenido nunca existe en claro fuera del cliente.

Funcionalidades implementadas. Las tres familias de requisitos funcionales del enunciado (RF1 gestión jerárquica, RF2 transferencia completa, RF3 acceso por primitivas) con su semántica declarada explícitamente (Sección 9), más locks con *fencing*, auditoría, rebalanceo de nodos y compactación de archivos anexados — funcionalidad que el enunciado no exige pero que el equipo agregó para que el sistema sea usable en la práctica.

Limitaciones del alcance (declaradas desde la Fase I, Sección 11 de ese documento, y mantenidas en la implementación):

- Sin compartición de archivos entre usuarios (no lo pide el enunciado).
- Sin semántica POSIX completa, ni enlaces simbólicos, ni archivos dispersos (*sparse*).
- Un único grupo Raft: la escala de metadatos queda acotada a lo que soporta ese grupo.
- El nodo de metadatos es de confianza: conoce la clave que envuelve la clave de cifrado de cada archivo.
- Sin *erasure coding* ni replicación geográfica entre regiones.

---

5. Diseño original (Fase I individual — Gloria Yuliana Peña Ibargüen)

Antes de implementar, cada integrante diseñó de forma independiente una propuesta completa para DFSha (Fase I, 40 % de la nota). El equipo comparó las tres propuestas (Sección 7) y adoptó la de **Gloria Yuliana Peña Ibargüen** como base. Esta sección resume esa propuesta original tal como fue entregada, antes de los ajustes de implementación.

5.1 Visión y supuestos

DFSha se diseñó desde el inicio con un plano de control (3 nodos de metadatos replicados con Raft) separado de un plano de datos (≥ 5 nodos de datos con bloques inmutables, cifrados y replicados $\times 3$). Los supuestos declarados: fallos *crash-stop* y de red (no bizantinos), corrección independiente de relojes sincronizados (se usan versiones y *fencing tokens*), y una escala objetivo de cientos de usuarios y 10^5–10^6 bloques de metadatos (caben en memoria del líder).

5.2 Casos de uso

La propuesta definió 10 casos de uso (autenticación, gestión de directorios, carga, descarga, abrir/leer, escribir, bloquear, borrar, gestionar nodos, detectar fallo y re-replicar),

cada uno vinculado a un requisito funcional o no funcional concreto del enunciado — tabla completa en el documento de Fase I.

5.3 Arquitectura propuesta

Componente	Plano	Responsabilidad
CLI/SDK `dfsha`	Cliente	Cifrado AES-256-GCM, partición en bloques, hash, E/S paralela, caché de manifiestos
DNS + NGINX	Control	Nombre estable `meta.dfsha.internal`, TLS, balanceo; no líderes responden 307
API de metadatos (FastAPI)	Control	Autenticación, espacio de nombres, transacciones, locks, block tokens
Raft + estado replicado	Control	Log replicado (quórum 2/3): árbol, manifiestos, ubicaciones, locks, usuarios
Monitores del líder	Control	Heartbeats, detección de caídas, re-replicación, rebalanceo, GC
Nodo de datos	Datos	Bloques inmutables `<block_id>.blk`, valida tokens, replica en pipeline, *scrubbing*

Decisiones clave declaradas desde la Fase I: metadatos con Raft (no un *NameNode* único) para eliminar el punto único de falla; los datos nunca pasan por el líder, de modo que el throughput crece con el número de nodos de datos; direcciones nunca fijas en el cliente (transparencia de localización, RNF7); y una réplica por zona de disponibilidad para tolerar la pérdida de una zona completa.

5.4 Semántica y consistencia propuestas

La propuesta ya fijaba, antes de escribir una sola línea de código: operaciones de escritura x/w/a/r+; visibilidad *close-to-open*; una escritura se considera confirmada cuando el commit devuelve 200, lo que implica manifiesto aceptado por quórum Raft **y** cada bloque con ≥ 2 réplicas verificadas por hash en ≥ 2 zonas; concurrencia optimista por defecto (CAS por versión, con *rebase* si los bloques modificados no se intersectan) y lock() pesimista opcional con *lease* de 30 s y *fencing token*.

5.5 Seguridad propuesta

Argon2id + JWT EdDSA para usuarios; mTLS con CA privada entre nodos; autorización por prefijo de ruta (/users/<uid>) más *block tokens* Ed25519 de corta vida para el plano de datos; cifrado de sobre en el cliente (DEK por archivo, AES-GCM por bloque, KEK en el servicio de metadatos); auditoría de solo-anexar.

Esta propuesta, con el nivel de detalle mostrado arriba (incluye además 10 experimentos de rendimiento previstos, un catálogo completo de errores y 4 escenarios de razonamiento obligatorio resueltos por escrito), fue la base que el equipo llevó a la Fase II.

6. Arquitectura consensuada final

La arquitectura final del equipo **es, en su mayor parte, la propuesta de Gloria sin cambios estructurales** — la comparación formal de la Sección 7 explica por qué se descartaron las otras dos. Los cambios reales frente a esa propuesta están documentados uno por uno en la Sección 7.2 (evolución del diseño); aquí se describe el resultado final, ya implementado y verificado en AWS.

6.1 Diagrama de componentes (ver diagrama de portada)

El plano de control son 3 nodos MDS en consenso Raft real (biblioteca PySyncObj) sobre túneles TLS 1.3 con autenticación mutua por certificado (mTLS) — Raft por sí solo solo habla TCP plano, así que cada MDS expone su propio túnel. El plano de datos son 4 nodos DN (el diseño original pedía ≥ 5; el laboratorio de AWS Academy limitó el número de instancias disponibles, decisión documentada como ADR en la Sección 7.2) con bloques inmutables cifrados, replicados en cadena.

6.2 Diagrama de despliegue (arquitectura física real en AWS)

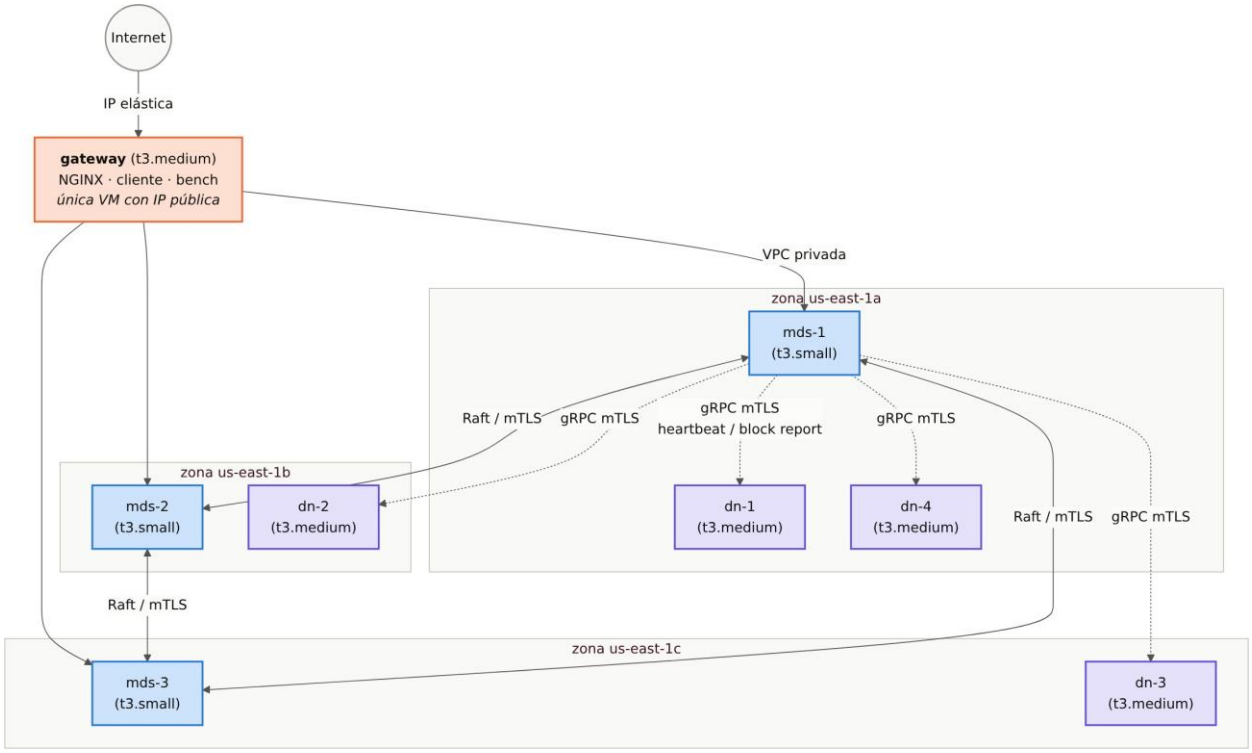


Figura 2: Diagrama de despliegue físico en AWS Academy: 8 VM reales en 3 zonas de disponibilidad

Justificación de las decisiones técnicas que definen esta arquitectura final:

- **Raft real (PySyncObj) en vez de escribirlo desde cero.** Implementar un algoritmo de consenso correcto, con todas sus pruebas de linealizabilidad, no era abordable con calidad

en el tiempo de la Fase II; se usó una biblioteca probada y se adaptaron sus límites (ver 4.1 y 4.13 en la Sección 7.2).

- **gRPC para datos, REST para control.** El plano de datos necesita *streaming* bidireccional y mensajes binarios compactos (gRPC sobre HTTP/2); el plano de control es naturalmente petición/respuesta y se beneficia de que REST sea simple de depurar y de describir con OpenAPI.
- **Ningún middleware de mensajería (Kafka/RabbitMQ).** Las tareas asíncronas (replicar, borrar) viajan como respuesta al *heartbeat* periódico del nodo de datos, evitando operar un componente adicional sin necesidad real.
- **El MDS nunca transporta el contenido del archivo.** Solo decide ubicaciones y metadatos; el cliente habla directo con los DN, así que el throughput agregado del sistema crece con el número de nodos de datos y no con la capacidad de un único nodo de control.

---

7. Evolución del diseño

7.1 Comparación formal de las tres propuestas individuales

Antes de implementar, el equipo construyó una matriz de riesgo ponderada para decidir, de forma objetiva, qué propuesta adoptar como base.

Criterio	Pregunta que responde	Peso
Buen diseño	¿Cumple RF1–RF3 y RNF1–RNF7, con garantías claras?	30 %
Alcanzable	¿Tres personas pueden implementarlo y probarlo en la Fase II?	40 %
Viable	¿Corre en AWS Academy y se opera sin servicios pesados?	30 %

Propuesta	Buen diseño	Alcanzable	Viable	Total ponderado
Gloria	4	4	4	**4,0**
J. Sebastián	3	2	2	2,3
Juan José	5	3	3	3,6

La propuesta de Juan José obtuvo la mejor nota en “buen diseño” puro (observabilidad completa, colas FIFO con *fencing* más robustas), pero perdió puntos en “alcanzable” y “viable”: proponía 12 VM con dos balanceadores HAProxy y un stack completo de observabilidad. La de J. Sebastián dependía de Consul, Keycloak y Vault — piezas operativas adicionales que el equipo no podía justificar asegurar y operar en el tiempo y los límites de AWS Academy. La de Gloria resultó la más equilibrada y fue la adoptada, incorporando puntualmente elementos de las otras dos donde mejoraban el diseño sin añadir piezas nuevas que operar (por ejemplo, el *pipeline* de replicación de Juan José, o su cola FIFO con *fencing* para locks).

7.2 Cambios frente al diseño original y su impacto

#	Diseño original (Fase I)	Implementado en Fase II	Justificación	Impacto en la implementación
1	Raft "genérico"	Biblioteca **PySyncObj**; métodos deterministas; *heartbeats* fuera del log (en memoria); modo *batch* desactivado	Escribir Raft desde cero no era abordable con calidad en el plazo	Operación de metadatos bajó de ~100 ms a ~5 ms al desactivar el *batch*
2	mTLS "en Raft" (afirmado, sin mecanismo)	`raft_tls.py`: cada MDS expone un túnel TLS 1.3 con certificado de cliente obligatorio; Raft escucha solo en loopback	PySyncObj solo habla TCP plano; había que cumplir mTLS sin tocar la biblioteca	Nuevo módulo dedicado (~150 líneas) que no existía en el diseño
3	DEK "envuelta por el MDS"	El cliente genera la DEK y la envía por TLS en el commit; el MDS la guarda envuelta y la devuelve desenvuelta por TLS en `open`	El diseño original entregaba al cliente una DEK que no podía descifrar sin el paso inverso	Cambio en el protocolo de transacción, no en el modelo de confianza
4	`content_sha256` del archivo completo	SHA-256 de la lista de hashes de cada bloque	Se recalcula tras un anexo o una modificación parcial sin releer el archivo completo	Verificación de integridad más barata en escrituras parciales
5	AES-GCM "por bloque"	AES-GCM por **segmentos de 64 KiB** dentro de cada bloque	Una sola etiqueta por bloque de 64 MiB obligaría a descargar el bloque completo para leer 1 byte	Habilitó lectura de rangos (`read(off, n)`) sin penalidad de red
6	Cadena de replicación simple	*Pipeline* con refuerzo: si un eslabón cae o tarda > 1 s, el primario empuja el bloque completo a los	Una cadena pura se bloquea con un eslabón lento	Evita que una réplica lenta bloquee el commit, sin publicar nada a medias

#	Diseño original (Fase I)	Implementado en Fase II	Justificación	Impacto en la implementación
		nodos que faltan		
7	— (no contemplado)	`raft_safety.py`: envuelve el *journal* de PySyncObj para que un `AppendEntries` atrasado nunca borre entradas ya aplicadas	Bug real de la biblioteca, detectado corriendo el experimento de caída repetida del líder (E7) bajo carga — violaba la regla §5.3 de Raft	Módulo nuevo; sin él, un seguidor podía perder entradas confirmadas
8	G2 (“`open` ve el último commit”)	`GET /v1/files/{path}` pasa por `barrier()` de Raft antes de leer, igual que `ls`/`stat`	Bajo carga, un `open()` justo después de un commit 200 podía leer una versión anterior (lectura no linealizable) — defecto real encontrado y corregido	Corrección de un bug de linealizabilidad, no un cambio de diseño
9	≥ 5 nodos de datos	4 nodos de datos	Límite de instancias disponibles en el laboratorio de AWS Academy	R=3 sigue siendo alcanzable con 4 nodos; documentado explícitamente como ajuste de infraestructura, no de diseño

Impacto general. Ningún cambio alteró las garantías prometidas en la Fase I (Sección 9 de este informe); todos fueron correcciones de implementación (bugs reales encontrados y corregidos bajo carga) o adaptaciones a límites concretos de la biblioteca de Raft o del laboratorio. La tabla completa de 18 decisiones vive en `docs/DISENO_FASE2.md` del repositorio.

---

8. Diseño e implementación

8.1 Estructura general

```
dfsha/
├── common.py        cifrado, tokens, hashes, catálogo de errores (compartido)
├── mds/
```

```
│   ├── state.py      máquina de estados replicada con Raft (el "cerebro")
│   ├── server.py     API REST (FastAPI): autenticación, txn, 307 al líder
│   ├── raft_tls.py   túneles TLS/mTLS para el tráfico de Raft
│   └── raft_safety.py salvaguarda del journal de Raft
├── dn/
│   └── server.py     nodo de datos: gRPC, replicación en cadena, scrubbing
└── client/
    ├── sdk.py         SDK (clase DFSha, Handle)
    └── cli.py         interfaz de línea de comandos
```

8.2 Módulos y responsabilidades

`dfsha/mds/state.py` es el corazón del sistema: una máquina de estados cuyos métodos de escritura están decorados `@replicated` (PySyncObj), de modo que cada cambio (crear un directorio, abrir una transacción, confirmar un commit, otorgar un lock) pasa por el log de Raft y se aplica igual en los 3 MDS. Implementa el árbol de directorios (RF1), las transacciones `OPEN→PREPARED→COMMITTED/ABORTED`, el CAS por versión con *rebase*, los locks S/X con *fencing*, la compactación de anexos y el recolector de basura.

`dfsha/mds/server.py` traduce HTTP a llamadas sobre esa máquina de estados: autentica el JWT, resuelve a qué usuario pertenece cada ruta, y si el nodo que recibe la petición no es el líder Raft, responde `307` con la ubicación del líder en vez de intentar escribir. También corre los monitores en segundo plano: detecta nodos `DEAD`, ordena re-replicación, rebalancea bloques entre nodos con distinta carga y limpia huérfanos.

`dfsha/dn/server.py` implementa el servicio gRPC `DataNode`: `PutBlock` (sube un bloque, lo reenvía en cadena a las siguientes réplicas — método `_chain`), `GetBlock` (lee un bloque o un rango de él), `CheckBlocks` (el líder verifica el hash de cada réplica antes de confirmar un commit) y el ciclo de vida interno de un bloque: se escribe como `.part`, se hace `fsync`, se verifica el hash y solo entonces se renombra a `.blk` — así un bloque a medias nunca es elegible para lectura.

`dfsha/client/sdk.py` y `cli.py` implementan el lado del cliente: parten el archivo en bloques, cifran cada segmento de 64 KiB con AES-256-GCM, suben los bloques en paralelo, y en la lectura descargan solo los bloques y segmentos necesarios según el rango pedido.

8.3 Interacciones entre componentes

Una escritura típica atraviesa: `cliente → POST /v1/txn (MDS líder)` → el líder asigna bloques y réplicas vía Raft → `cliente → PutBlock (gRPC, DN primario)` → el DN primario reenvía en cadena a las siguientes réplicas (`_chain`) → `cliente → POST /v1/txn/{id}/commit (MDS líder)`
→ el líder llama `CheckBlocks` a cada DN asignado antes de aceptar → si ≥ W réplicas verifican el hash, el commit se replica por Raft y responde `200`. Este flujo — descrito aquí en una frase — es exactamente la Figura 3 de la Fase I (Sección 5), implementada sin cambios de fondo.

---

9. APIs y comunicación

9.1 REST (plano de control, `/v1`)

Método	Ruta	Propósito
POST	`/v1/auth/login`, `/refresh`,	`/l`

	`ogout`	Autenticación y renovación de JWT
GET/POST/DELETE	`/v1/fs/{path}`	`ls`/`stat`, `mkdir`/`mv`, `rm`/`rmdir`
POST	`/v1/txn`, `/{id}/commit`, `/abort`, `/renew`	Ciclo de vida de una transacción de escritura
GET	`/v1/files/{path}?version=`	Manifiesto + ubicaciones + *block tokens* para lectura
GET	`/v1/blocks/{id}/locations`	Refresca réplicas vivas de un bloque
POST	`/v1/blocks/{id}/bad`	Un cliente reporta una réplica sospechosa (ver Sección 13 integridad)

Método	Ruta	Propósito
POST/GET/PUT/DELETE	`/v1/locks`	Adquirir, consultar, renovar, liberar un lock
GET/POST/DELETE	`/v1/admin/nodes`, `/v1/admin/blocks`	Estado del clúster, alta/retiro de nodos (rol `admin`)

9.2 gRPC (plano de datos)

`DataNode.{PutBlock, GetBlock, StoreBlock, ReplicateBlock, DeleteBlock, CheckBlocks}` — streaming de trozos cifrados, token de bloque en cada llamada. `Meta.{Heartbeat, BlockReport}` — cada nodo de datos reporta su estado cada 3 s y recibe, en la misma respuesta, cualquier orden pendiente del líder (replicar o borrar un bloque), evitando un canal de mensajería aparte.

9.3 Formato de intercambio

JSON sobre HTTPS para el plano de control (con esquema OpenAPI autogenerado por FastAPI, disponible en `/docs` de cada MDS); *Protocol Buffers* binarios sobre HTTP/2 para el plano de datos.

9.4 Flujo de comunicación — secuencia de una carga de archivo

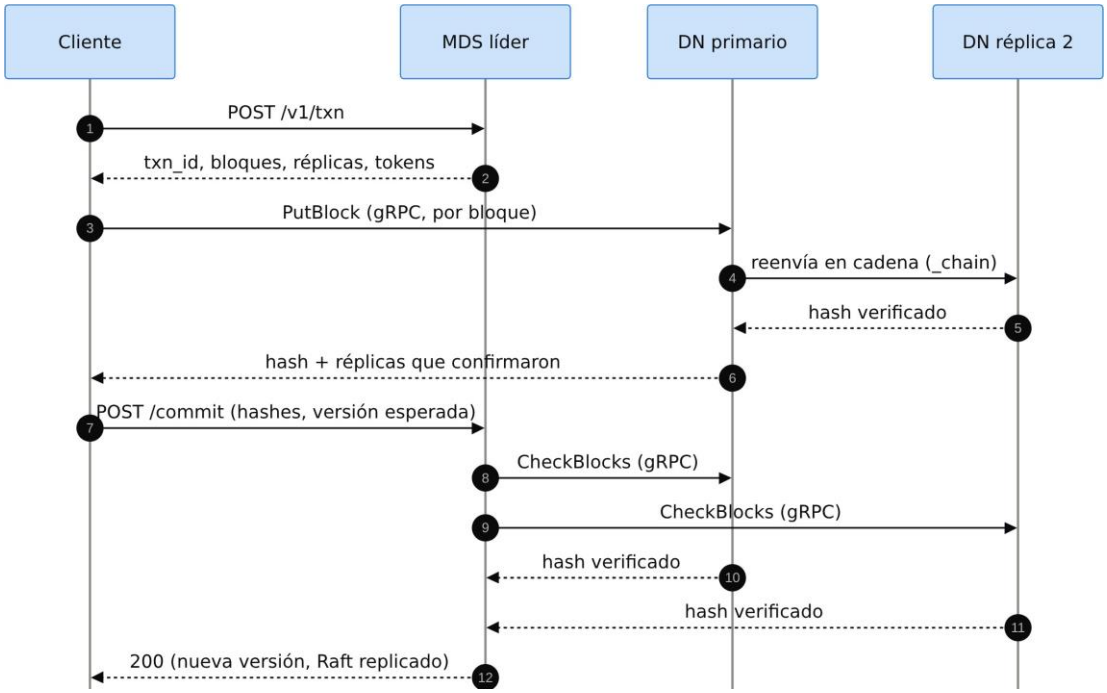


Figura 3: Secuencia de una carga de archivo: transacción, PutBlock en cadena, CheckBlocks y commit replicado por Raft

---

10. Algoritmos y lógica distribuida

10.1 Consenso para metadatos (Raft)

Los 3 MDS forman un grupo Raft (PySyncObj) con quórum 2 de 3: cualquier operación que modifique el árbol de directorios, un manifiesto, una ubicación de bloque o un lock pasa por el log replicado antes de aplicarse, dando linealizabilidad a las operaciones de nombres. El líder se reelige automáticamente si cae (ventana medida: 2.1–2.9 s, Sección 13); mientras tanto, lecturas sobre datos ya abiertos siguen funcionando porque no dependen del líder de metadatos. Una salvaguarda adicional (`raft_safety.py`) corrige un comportamiento real de la biblioteca: sin ella, un `AppendEntries` atrasado podía borrar entradas del *journal* que ya se habían aplicado, violando la regla de seguridad de Raft (§5.3 del paper original).

10.2 Coordinación de escrituras concurrentes (CAS optimista + *rebase*)

Cada commit lleva una `expected_version`. El líder aplica *compare-and-swap*: si la versión actual coincide, publica; si no, compara qué bloques modificó cada escritor desde la versión base — si los conjuntos son disjuntos, aplica un *rebase* y ambos commits terminan confirmados (versión final con los cambios de los dos); si se solapan, el segundo recibe `409 VERSION_CONFLICT` y sus bloques subidos quedan huérfanos para el recolector de basura. Esto se verificó experimentalmente en T3/E5: con bloques distintos, la tasa de conflicto se mantiene en 0–50 % incluso con 8 clientes concurrentes; sobre el *mismo* bloque, sube hasta 87.5 % — exactamente el comportamiento esperado.

10.3 Exclusión mutua pesimista (`lock()` con *fencing*)

Para los casos que sí necesitan exclusión real, `lock()` ofrece modo compartido (S) o exclusivo (X), sobre un archivo completo o un bloque específico, con un *lease* de 30 s renovado cada 10 s y guardado en el log de Raft. Cada concesión incluye una época monótona (*fencing token*): si un titular cuyo *lease* expiró reaparece e intenta escribir con la época vieja, recibe `409 STALE_FENCING` en vez de corromper silenciosamente el estado — el mecanismo clásico descrito por Kleppmann para evitar el problema del "proceso zombi" en sistemas distribuidos.

10.4 Distribución de bloques y balanceo de carga

Para cada bloque se eligen 3 nodos de datos con el algoritmo *power of two choices*: se muestrean 2 candidatos al azar por espacio libre y se elige el menos cargado, exigiendo nodos y zonas de disponibilidad distintas. Esto evita tanto la concentración de bloques en pocos nodos (el problema de elegir siempre "el más libre" de forma determinista) como el costo de examinar todos los nodos en cada asignación. El nodo primario de cada bloque rota entre las réplicas asignadas para repartir la carga de escritura de entrada.

10.5 Replicación en *pipeline* con refuerzo

El cliente sube un bloque una sola vez, al nodo primario; este lo reenvía en cadena (`_chain`) a las siguientes réplicas mientras aún lo está recibiendo, en vez de esperar a tenerlo completo — así el tiempo total de replicación no es la suma de las tres subidas, sino aproximadamente la subida más el salto adicional de red entre nodos. Si un eslabón de la cadena cae o tarda más de 1 s, el primario detecta la ruptura y empuja el bloque completo directamente a los nodos que faltan, garantizando el quórum de escritura (W=2) sin bloquearse indefinidamente en un nodo lento.

10.6 Tolerancia a fallos

Mecanismo	Umbral	Efecto
Heartbeat de cada DN al líder	cada 3 s	Base de toda la detección de fallos
`SUSPECT`	sin *heartbeat* > 10 s	El nodo deja de recibir nuevas escrituras
`DEAD`	sin *heartbeat* > 30 s	Se retira de las ubicaciones; arranca re-replicación priorizada
Scrubbing de integridad en	cada 6 h + verificación en cada lectura	Detecta corrupción silenciosa (*bit rot*) antes de que un lector la encuentre

La re-replicación prioriza los bloques con menos réplicas vivas, evita repetir una zona ya cubierta, y respeta un límite de ancho de banda por nodo (`DFSHA_REPL_MBPS`) para no saturar la red durante la recuperación — medido en el experimento E8 (Sección 13): el tiempo hasta restaurar R=3 crece de forma aproximadamente lineal con el volumen perdido (2.0 s para 16 MB, 6.7 s para 96 MB).

---

11. Despliegue

11.1 Infraestructura utilizada

8 máquinas virtuales de AWS Academy, Ubuntu Server 24.04 LTS, en 3 zonas de disponibilidad reales de `us-east-1` (a/b/c), dentro de una VPC con un único grupo de seguridad (`dfsha-sg`) que permite todo el tráfico entre sus propios miembros y SSH solo desde la IP del equipo. Cada servicio corre en un contenedor Docker, construido desde una sola imagen (`dfsha:latest`) que cambia de rol según el comando con el que se arranca.

VM	Tipo	Zona	Disco	Rol
gateway	t3.medium	us-east-1a	20 GiB	NGINX (IP elástica), cliente, *benchmarks*
mds-1 / mds-2 / mds-3	t3.small	a / b / c	10 GiB	Metadatos (Raft)
dn-1 / dn-2 / dn-3 / dn-4	t3.medium	a / b / c / a	20 GiB (EBS cifrado)	Datos

11.2 Configuración de nodos y herramientas de ejecución

El despliegue completo está automatizado en `deploy/aws/deploy_all.sh`, ejecutado desde el *gateway* (única VM con IP pública) en 6 etapas: preparar Docker e imagen, descubrir las IP privadas reales de cada VM, generar y distribuir la PKI (certificados mTLS por nodo), enviar el código, arrancar los contenedores con el rol correcto (`ROLE=mds|dn|gateway`, `MDS_ID`/`DN_ID` según corresponda) y verificar con una escritura/lectura de prueba. Documentado paso a paso, incluyendo la creación de las VM desde la consola de AWS, en `docs/DESPLIEGUE_PASO_A_PASO.md`.

11.3 Requisitos técnicos

Python 3.13, Docker, acceso SSH con el par de claves `dfsha-key` creado específicamente para este despliegue (no el `vockey` genérico de AWS Academy), y el grupo de seguridad configurado antes de lanzar las instancias.

12. Estrategia de pruebas

El proyecto tiene **dos suites de pruebas independientes** que ejercitan exactamente los mismos 7 escenarios mínimos exigidos (§5.2 del enunciado), analizadas directamente de los archivos fuente:

- **`tests/test_t1_*.py` ... `test_t9_*.py`** (ejecutadas con `pytest`): levantan un clúster de 3 MDS + 5 DN como **procesos reales** en la máquina local, comunicándose por TCP con TLS/mTLS — no en memoria. Objetivo: validación rápida y repetible en cualquier máquina, incluso sin AWS.
- **`deploy/aws/pruebas_aws.py`** (ejecutado desde el *gateway*, contra el clúster real de 8 VM): mismas 7 pruebas mínimas, pero inyectando fallos **reales** (`deploy/aws/chaos_all.sh`: `kill -9` real de un proceso, `iptables` real para partición de red, `tc netem` real para latencia, corrupción real de 1 byte en un `.blk`) y usando SSH para inspeccionar el disco de cada nodo. Objetivo: la verificación que el enunciado exige explícitamente ("no basta

una simulación local monolítica").

12.1 Casos de prueba, objetivo y metodología (de `pruebas_aws.py`)

Función	Objetivo	Metodología
`t1()`	Validación funcional de carga/recuperación	Archivo aleatorio de tamaño configurable (`--size-mb`), `put`, se borra el original, `get`, se compara SHA-256; se inspecciona por SSH el número real de archivos `.blk` en 3 DN
`t2()`	Validación funcional de RF3	Secuencia `x`→`EXISTS`, `r+` con verificación de *copy-on-write* (solo cambia el `block_id` del bloque tocado), `r+` más allá de EOF → `400`, lectura cruzando bloques, visibilidad *close-to-open* con dos clientes, anexo, reemplazo
`t3()`	Validación de concurrencia	Dos hilos (`threading`) escriben simultáneamente: mismo bloque (uno gana, uno `409`), bloques distintos (ambos confirman por *rebase*), anexos concurrentes, lock `X` por bloque (el otro recibe `423`)
`t4()`	Validación distribuida — fallo de nodo	`chaos kill` **real** a un DN mientras se lee (debe seguir íntegro); espera a que se marque `DEAD`; espera la re-replicación real a R=3; repite con el líder de metadatos
`t5()`	Validación distribuida — consistencia	`chaos delay/partition` real sobre un DN mientras se escribe; detiene todos menos 1 DN y verifica `503`/`404` sin estado parcial
`t6()`	Validación de seguridad	Intentos de acceso cruzado entre usuarios (`403`), JWT falso/vencido (`401`), `openssl s_client` real contra TLS 1.2 (debe rechazar), captura `tcpdump` real del tráfico cliente→DN (0 texto claro), inspección

		por SSH del contenido de los `.blk`, corrupción real de 1 byte y verificación de que la lectura detecta y recupera
`t7()`	Métricas de rendimiento	≥ 5 repeticiones por condición (tamaño 1/10/`--perf-mb` MB × paralelismo 1/4, más 1 y 4 clientes concurrentes leyendo); calcula media, desviación y p95; exporta CSV

Cada función usa `check()` (equivalente a una aserción) para fallar explícitamente con un mensaje claro si el comportamiento observado no coincide con la garantía esperada, y `ev()` escribe cada paso con marca de tiempo en `evidencias_aws/<Tn>.txt` — así la evidencia no es solo "pasó/falló" sino el registro completo de lo que ocurrió en el clúster real.

---

13. Resultados experimentales

13.1 Resultados funcionales y de fallos — clúster real en AWS (8 de noviembre de 2026)

T1–T7: 7 de 7 pruebas superadas contra el clúster real.

Prueba	Resultado observado
T1 — Carga y recuperación	256 MB → 64 bloques en 3 DN distintos; `put` 79.1 MB/s, `get` 109.5 MB/s; SHA-256 idéntico
T2 — RF3	Las 10 comprobaciones de semántica pasaron; copy-on-write confirmado (solo 1 de 4 `block_id` cambió tras un `r+`)
T3 — Concurrencia	Mismo bloque: 1×`200` + 1×`409` `VERSION_CONFLICT`. Bloques distintos: 2×`200` (rebase). Lock `X`: el segundo escritor del mismo bloque recibe `423`
Prueba	**Resultado observado**
T4 — Fallos	`kill` real a un DN: lectura OK en 0.60 s, `DEAD` detectado a los 28 s, R=3 restaurado en 409 s. `kill` real al líder: 2.75 s de indisponibilidad, nuevo líder, escrituras siguen con quórum 2/3
T5 — Consistencia	Partición real de un DN: escritura/lectura OK con W=2. Con solo 1 DN vivo: `503` `INSUFFICIENT_REPLICAS` y `404` (nada parcial publicado)
T6 — Seguridad	401/403 en los 7 casos probados; TLS 1.3 aceptado, TLS 1.2 rechazado (`openssl s_client` real); captura `tcpdump` de 470

	paquetes reales con 0 ocurrencias de texto claro; corrupción real de 1 byte detectada y recuperada sin pérdida
T7 — Rendimiento	Ver 13.2

13.2 Rendimiento — clúster real en AWS (`evidencias_aws/rendimiento.csv`)

Condición	Operación	Tamaño	Media	p95	MB/s
1 MB, paralelo=1	put / get	1 MB	0.109 s / 0.064 s	0.113 / 0.067	9.2 / 15.6
10 MB, paralelo=4	put / get	10 MB	0.227 s / 0.147 s	0.250 / 0.179	44.0 / 67.9
100 MB, paralelo=1	put / get	100 MB	2.509 s / 1.104 s	2.608 / 1.217	39.9 / 90.6
100 MB, paralelo=4	put / get	100 MB	1.396 s / 0.908 s	1.424 / 1.223	**71.6 / 110.1**
4 clientes concurrentes	get agregado	100 MB c/u	3.752 s	3.782	106.6 agregados

Interpretación. El paralelismo de 4 flujos casi duplica el throughput de escritura frente a 1 flujo (71.6 vs. 39.9 MB/s a 100 MB) — consistente con la hipótesis de la Fase I de que el rendimiento se acerca al de la red a medida que el tamaño del archivo diluye el costo fijo por transacción. Con 4 clientes leyendo el mismo archivo en paralelo, el throughput agregado (106.6 MB/s) es comparable al de un solo cliente con 4 flujos (110.1 MB/s) — indicio de que, en esta topología de 4 nodos de datos, el cuello de botella ya es compartido entre clientes antes de que la concurrencia adicional aporte más.

13.3 Resultados de la suite local (procesos separados en un solo host, referencia)

La suite local (`pytest tests -q`) complementa la de AWS con 3 pruebas adicionales que el mínimo del enunciado no exige (T0 journal de Raft, T8 borrado/huérfanos, T9 fidelidad completa, T10 *gateway* NGINX) y con 10 experimentos de rendimiento (E1–E10) bajo condiciones controladas de un solo host — útiles para aislar variables (p. ej. costo de TLS/AES: E10 mide un sobrecosto de ~34 % en escritura) que en AWS se mezclan con la variabilidad real de la red. Última corrida: **38 pruebas superadas, 2 omitidas** (T10 se omite si la máquina no tiene `nginx` instalado). Detalle completo en `evidencias/RESUMEN.md` y gráficas en `evidencias/graficas/`.

---

14. Decisiones de ingeniería

Decisión	Por qué se eligió	Alternativa descartada y por qué	Trade-off aceptado
Raft con PySyncObj	Consenso probado, sin escribirlo desde cero	Implementación propia de Raft	Depender de los límites de una biblioteca de terceros (ver 7.2, filas 1 y 7)
gRPC para datos, REST para control	Bloques inmutables	+ copy-on-write Cifrado en el cliente	(DEK por archivo)

CA privada propia para mTLS	Cada protocolo donde rinde más: streaming eficiente vs. depuración simple	Un solo protocolo para todo (propuesta de J. Sebastián)	Dos pilas de red que mantener en vez de una
	Publicación atómica trivial; réplicas y recuperación simples	Actualización en el lugar (*in situ*)	Modificar 1 byte reescribe un bloque completo
	Los nodos de datos nunca ven texto claro; rotar la KEK es barato	Cifrar en el nodo de datos, o solo cifrar el disco	El servicio de metadatos queda en la base de confianza (conoce la KEK)
	Sin dependencia operativa de un servicio externo	PKI gestionada por Vault (propuesta de J. Sebastián)	Gestión manual de la vigencia y distribución de certificados
W=2 de R=3	Equilibrio latencia/durabilidad: tolera 1 fallo sin bloquear escrituras	W=3 (más durable, más lento) o W=1 (más rápido, menos seguro)	Una escritura puede confirmar con una réplica todavía propagándose en segundo plano
Power of two choices para colocación	Evita tanto la concentración de bloques como el costo de examinar todos los nodos	Colocación puramente aleatoria, o siempre "el nodo más libre"	Ligera sub-optimalidad frente a un algoritmo exhaustivo, a cambio de O(1) por asignación
4 nodos de datos en vez de ≥ 5	Límite real de instancias del laboratorio de AWS Academy	Mantener 5 y usar contenedores adicionales en VM existentes	R=3 sigue siendo alcanzable con 4 nodos; menos margen ante una segunda caída simultánea

Ventajas obtenidas de este conjunto de decisiones: un sistema que corre con procesos reales en red, con todas las garantías de consistencia y seguridad prometidas desde la Fase I, verificado tanto en local como en un despliegue real de 8 máquinas — sin depender de ningún servicio operado por terceros fuera de AWS mismo.

15. Limitaciones

- **Un único grupo Raft.** La escala de metadatos queda acotada a lo que soporta ese grupo; una mejora futura evidente es particionar el espacio de nombres por usuario entre varios grupos Raft independientes.
- **El servicio de metadatos es de confianza.** Conoce la clave que envuelve la clave de cifrado de cada archivo (KEK) y la entrega al cliente por TLS; un MDS comprometido podría,

en teoría, desenvolver cualquier DEK.

- **TLS mínimo en gRPC depende de la biblioteca.** `grpcio` para Python no expone un mínimo de versión configurable; negocia 1.3 en la práctica, pero no se puede forzar a nivel de API como sí se hace en REST.
- **Sin compartición de archivos entre usuarios**, semántica POSIX completa, archivos dispersos, *erasure coding* ni replicación geográfica — todas declaradas explícitamente desde la Fase I y mantenidas sin cambios.
- **Auditoría no unificada.** Cada líder MDS anexa a su propio archivo de auditoría; no existe (todavía) un log de auditoría consolidado entre los tres nodos.
- **4 nodos de datos, no ≥ 5.** Por el límite de instancias del laboratorio; reduce el margen ante una segunda falla simultánea a la ya tolerada.
- **Pruebas de rendimiento en AWS con un único cliente/gateway.** El experimento de escalabilidad entre 3/5/7 nodos de datos (E9) se corrió solo en local; en AWS no se repitió por el límite de instancias disponibles.

16. Trabajo futuro

- **Particionar el grupo Raft por usuario** para escalar la capacidad de metadatos más allá de lo que soporta un único grupo.
- **Mover la KEK fuera del MDS** (por ejemplo, a un KMS gestionado) para que el servicio de metadatos deje de ser parte de la base de confianza del cifrado.
- **Erasure coding** en lugar de replicación simple ×3 para bloques fríos, reduciendo el costo de almacenamiento sin perder durabilidad.
- **Replicación geográfica** entre regiones de AWS para tolerar la pérdida de una región completa, no solo de una zona de disponibilidad.
- **Unificar la auditoría** de los tres MDS en un log consolidado y firmado.
- **Compartición de archivos entre usuarios**, hoy fuera de alcance por decisión explícita del enunciado, como extensión natural del modelo de autorización ya implementado.
- **Repetir en AWS, con más instancias si el laboratorio lo permite, el experimento de escalabilidad E9** (3→5→7 nodos de datos) para confirmar en red real la tendencia observada en local.

17. Conclusiones

El equipo diseñó, implementó y verificó un sistema de archivos distribuido que cumple los siete requisitos no funcionales y los tres requisitos funcionales del enunciado, con siete garantías de consistencia explícitas y comprobables, sobre una arquitectura que surgió de comparar formalmente tres propuestas individuales y adoptar, con justificación documentada, la más equilibrada entre buen diseño, viabilidad técnica y alcanzabilidad para tres personas en el tiempo disponible.

El cumplimiento de objetivos no se quedó en la promesa de diseño: las siete pruebas mínimas exigidas corrieron contra un clúster real de 8 máquinas virtuales en AWS Academy, con fallos inyectados de verdad — procesos matados, red particionada, un byte corrompido en disco — y el sistema se comportó exactamente como prometía el diseño de Fase I: ninguna lectura devolvió un dato corrupto o parcial, ninguna escritura concurrente se perdió en silencio, y cada falla simulada se recuperó de forma automática y medible.

El principal aprendizaje técnico del equipo fue que la distancia entre "diseñado para tolerar fallos" y "verificado tolerando fallos reales" es donde aparecen los defectos genuinos — como el bug de linealizabilidad de `open()` o el comportamiento inesperado del *journal* de Raft bajo caídas repetidas — que ninguna revisión de diseño, por cuidadosa que sea, encuentra sin ejecutar el sistema bajo carga real. El valor de esta solución no está solo en que funcione, sino en que cada garantía que afirma puede señalarse a una prueba concreta, con evidencia

capturada, que la demuestra.